

Estrutura de Cenas — EcoCat

Este documento descreve a estrutura de cenas (*scenes*) do projeto Godot **EcoCat**.

No Godot, todo o jogo é montado em **árvores de nós (nodes)**. Cada arquivo `.tscn` é uma *cena*: uma árvore de nós. Cenas podem ser **instanciadas** dentro de outras, permitindo reuso. A estrutura abaixo mostra quais cenas existem, qual é a principal e como os nós se organizam.

Cena principal

A cena executada ao iniciar o jogo é definida em `project.godot` :

```
run/main_scene = "res://scenes/Main.tscn"
```

Observação importante: a `Main.tscn` está quase vazia no editor — ela contém apenas o nó raiz com o script `scripts/main.gd`. Quase todo o conteúdo do jogo (jogador, lixeiras, resíduos, colisões do cenário e HUD) é **criado em tempo de execução por código**, dentro de `main.gd` (função `_ready()`). Por isso este documento apresenta duas visões: as cenas estáticas (o que está salvo nos `.tscn`) e a árvore em tempo de execução (o que existe na tela quando o jogo roda).

1. Cenas estáticas (arquivos `.tscn`)

`scenes/Main.tscn` — cena principal

```
Main (Node2D) [script: scripts/main.gd]
```

`player/Player.tscn` — o jogador (EcoCat)

```
Player (CharacterBody2D) [script: player/player.gd]
├─ CollisionShape2D (CircleShape2D, raio 14)
└─ Camera2D (zoom 1.4, suavização de posição ativada)
```

`items/WasteItem.tscn` — resíduo coletável

```
WasteItem (Area2D) [script: items/waste_item.gd]
└─ CollisionShape2D (CircleShape2D, raio 16)
```

items/RecycleBin.tscn — lixeira de reciclagem

```
RecycleBin (Area2D) [script: items/recycle_bin.gd]
└─ CollisionShape2D (RectangleShape2D, 72×76)
```

2. Árvore em tempo de execução

Quando o jogo roda, `scripts/main.gd` instancia as demais cenas e monta a seguinte árvore sob o nó `Main`:

```
Main (Node2D)
├─ WorldCollisions (Node2D) colisões do cenário
│   ├── BuildingCollision (StaticBody2D) × 4 (prédios)
│   ├── TreeCollision (StaticBody2D) × 7 (árvores)
│   └─ RecycleBinCollision (StaticBody2D) × 5
├─ RecycleBins (Node2D)
│   └─ RecycleBin (instância de RecycleBin.tscn) × 5
├─ WasteItems (Node2D)
│   └─ WasteItem (instância de WasteItem.tscn) × 5
├─ Player (instância de Player.tscn)
└─ CanvasLayer (HUD)
    ├── Label (instruções de jogo)
    ├── Label (moedas / item atual / sustentabilidade)
    └─ Label (mensagem de feedback)
```

O cenário visual (grama, ruas, prédios e árvores) é desenhado por código na função `_draw()` de `main.gd`, usando texturas de `textures/`.

3. Comunicação entre cenas (signals)

A `Main` não acessa os objetos diretamente: as cenas instanciadas avisam a `Main` por meio de *signals*, mantendo o acoplamento baixo.

Cena	Signal emitido	Tratado em <code>main.gd</code>
<code>WasteItem</code>	<code>picked_up</code>	<code>_on_waste_item_picked_up()</code>
<code>RecycleBin</code>	<code>discard_requested</code>	<code>_on_discard_requested()</code>

Fluxo do jogo:

1. O jogador encosta em um `WasteItem` → emite `picked_up` → a `Main` coloca o resíduo no inventário (um item por vez).

2. O jogador leva o resíduo até uma `RecycleBin` → emite `discard_requested` → a `Main` compara o tipo do resíduo com o tipo da lixeira:
- **correto:** ganha moedas e aumenta a sustentabilidade;
 - **errado:** perde sustentabilidade.

4. Resumo das cenas

Cena	Nó raiz	Papel no jogo
<code>Main.tscn</code>	<code>Node2D</code>	Cena principal; monta o mundo por código
<code>Player.tscn</code>	<code>CharacterBody2D</code>	Personagem controlável com câmera
<code>WasteItem.tscn</code>	<code>Area2D</code>	Resíduo coletável (5 tipos)
<code>RecycleBin.tscn</code>	<code>Area2D</code>	Lixeira de descarte (5 tipos)